

AI によるヘテロジニアス NUMA システムにおける負荷分散戦略の自動設計

概要

異種 NUMA システムにおいて、アプリケーション性能の最大化を目標として、AI を用いて負荷分散戦略（ワークロードをどの NUMA ノードへ割り当てるか）を自動設計する。従来の手動設計やルールベース手法と比べて、学習駆動の自動設計により、多様なワークロードへの適応性を高める。

背景

異種 NUMA システムとは

現代の異種マルチコアプロセッサでは、性能コア（P コア）と高効率コア（E コア）が同一チップ上に共存し、各コアは異なる NUMA（Non-Uniform Memory Access）ノードに結び付いている。この構造では、コアの割り当て方によってメモリアクセス遅延が大きく変化する。

負荷分散戦略の種類

本研究で扱う負荷分散戦略は 5 種類である。

戦略	動作
Packed	ワークロードをローカル側（性能コアに近い側）へ集中的に配置する
Scatter	ワークロードを複数の NUMA ノードへ分散配置する
HPO（Heuristic Priority Ordering）	性能コアを優先的に割り当てる
MPO（Memory-aware Priority Ordering）	メモリ親和性に基づいて優先的に割り当てる
SPO（Scheduling Priority Ordering）	スケジューリング優先度を協調的に制御する

既存研究の課題

既存研究では、以下の問題が明らかになっている。

- 単一の負荷分散戦略では、すべてのワークロードにおいて最適にならない
- Packed / Scatter はメモリページの配置を制御し、HPO / MPO / SPO はスレッドスケジューリングを制御するが、両者は長らく独立に設計されてきた
- ルールベース手法（POSM など）は平均的には優れた性能を示すが、特定のワークロードでは顕著な損失が生じる
- 機械学習ベース手法（HML-RF など）の平坦な 5 分類では、戦略間の構造関係を有効に活用できず、スイート間汎化に明らかな弱点がある

AI 自動設計の位置づけ

本研究は、AI に「ワークロード特性に応じて、どの負荷分散戦略を選択すべきか」を自動学習させ、ルールベース手法の限界を突破し、多様なワークロードへ柔軟に適応できるようにすることを目指す。また、人手によるパラメータ調整を不要にする。

研究目的

- 異種 NUMA システムにおける負荷分散戦略選択の自動化を実現する
- AI に基づく自動設計手法を開発・実装する
- ルールベース手法および機械学習ベース手法との性能比較を通じて、提案手法の有効性を検証する

研究内容

- 負荷分散戦略の性能データ収集
Sniper などのサイクル精度シミュレータを用い、多様なベンチマーク（PARSEC、NPB、SPLASH2）に対して 5 種類の負荷分散戦略をそれぞれ適用し、実行結果（実行時間、エネルギー消費、電力）を収集する。既存データセットを活用し、必要に応じて追加計測を行う。
- ワークロード特徴量の設計
負荷分散戦略の選択に有効な特徴量を設計する。候補は以下の通りである。
 - メモリアクセスパターン（局所性、帯域要求）
 - スレッド並列度
 - ノード間メモリアクセス比率
 - futex などの同期イベント頻度
- AI 自動設計手法の開発
以下の手法を比較・検討する。
 - 階層構造**：まず大まかな戦略カテゴリを予測し、その後にサブクラス内で細粒度の選択を行う二層構造
 - コスト感度学習**：高損失リスクのサンプルに大きな重みを与える
 - 不確実性回避**：予測信頼度が低い場合に、ルールベース手法（POSM）へ戻す
 - 自動パラメータ設計**：ベイズ最適化などを用いて閾値や重みを自動調整する
- スイート間汎化性能の検証
1 つのベンチマークスイート（PARSEC、NPB、SPLASH2）をテストセットとし、残りを学習セットとする LOSO（Leave-One-Suite-Out）評価を行い、構築した手法の汎化能力を検証する。

期待される成果

- ルールベース手法（POSM）を上回る負荷分散戦略選択精度
- 多様なワークロードへ自動適応できる設計手法の確立
- スイート間汎化に関する知見の整理と、設計指針の策定

課題

- NUMA シミュレーション環境および多様なワークロード実行環境に依存するため、環境構築コストが高い
- 機械学習モデルの訓練には十分に大きなデータセットが必要であり、データ収集コストを無視できない
- 自動設計手法の意思決定過程がブラックボックス化しやすく、解釈性の向上が重要な課題である

実施ステップ

- まず、既存の性能データおよび特徴量設計を参照し、データ収集・整理フレームワークを構築する
- 次に、階層構造や不確実性回避などの設計案を比較し、有効な手法を選定する
- 最後に、LOSO 評価によって汎化性能を検証し、設計手法の有効性を確認する